

PAR Laboratory Assignment

Lab 4: Parallel task decomposition implementation and
analysis

Mario Acosta, Eduard Ayguadé, Rosa M. Badia, Jesús Labarta, Pau Ferrer,
Daniel Jiménez-González, Pablo Vizcaino, and Gladys Utrera

Spring 2025–2026



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Departament d'Arquitectura de Computadors

Contents

1	Experimental Setup	5
1.1	Laboratory files	5
1.2	Mandelbrot parameters	6
1.3	Compilation	6
1.4	Execution	6
1.5	Analysis	6
2	Iterative task decomposition	7
2.1	Implementation	7
2.2	Execution	8
2.3	Correctness	8
2.4	Performance analysis	8
3	Recursive task decomposition	9
3.1	Implementation	9
3.2	Execution	9
3.3	Correctness	10
3.4	Performance Analysis	10
A	Laboratory deliverable information	11
A.1	Individual results for each parallel strategy	11
A.2	Final comparison between all the parallel strategies	11
A.3	Evaluation of English as a third language	12
A.4	Submission of the deliverable and source codes to Atenea	12

Note. Chapters 2 and 3 of this document correspond each to two laboratory sessions (2 hours per session).

Important!

Your objective is to implement (in some cases only complete), execute, and analyse iterative and recursive task decompositions of Mandelbrot. Finally, you will compare the best iterative and recursive versions.

Annex A describes the laboratory document that you will need to elaborate and upload to Atenea together with all your C source codes.

Chapter 1

Experimental Setup

1.1 Laboratory files

This laboratory assignment will be done in four sessions of two hours each. All the files necessary to carry out this laboratory are available in a compressed file at `/scratch/nas/1/par0/sessions/lab4.tar.gz` located in `boada.ac.upc.edu`.

Uncompress the aforementioned file in your home directory with these command lines:

```
> cd
> tar -zxvf ~par0/sessions/lab4.tar.gz
```

You will find the following files in the `lab4` directory:

- Makefile.

- Source codes:

mandel-seq-iter.c: Iterative sequential code.

mandel-seq-rec.c: Recursive sequential code.

mandel-omp-iter.c: *Incomplete* coarse grain iterative OpenMP parallel code that tries to implement the first parallel strategy we analysed with Tareador during Lab 3.

mandel-omp-rec.c: *Incomplete* recursive OpenMP parallel code that tries to implement the leaf parallel strategy we analysed with Tareador during Lab 3.

- Scripts:

submit-seq.sh: Script to execute the sequential code and useful to obtain the expected outputs of Mandelbrot. You should use the outputs of the iterative and recursive sequential code to check the correctness of your parallel programs.

submit-omp.sh: Script to execute an OpenMP parallel code and useful to check the correctness of your programs and find out the performance of your parallel program for a particular number of threads.

submit-strong-omp.sh: Script to perform the strong scalability analysis of OpenMP parallel codes that generates two output files with the information of the speedup and elapsed time of the Mandelbrot function:

speedup-partial-program-hostname.txt: Speedup file.

elapsed-partial-program-hostname.txt: Elapsed execution times file used to fill in Table [A.2](#).

submit-strong-extrae.sh: Script to perform the Modelfactors analysis of OpenMP parallel codes. Remember that its execution also generates a set of Paraver traces within the Modelfactors directory.

1.2 Mandelbrot parameters

The arguments to be pass to the executables can be seen by running the command `mandel -help`:

```
> ./mandel-seq-iter -help
Usage: ./mandel-seq-iter [-o -h -d -i maxiter -c x0 y0 -s size]
  -o to write computed image (mandel_image.jpg) and histogram \
      (mandel_histogram.out if -h indicated) to disk (default no file generated)
  -h to produce histogram of values in computed image (default no histogram)
  -d to display computed image (default no display)
  -i to specify maximum number of iterations at each point (default 100)
  -c to specify the center x0+iy0 of the square to compute
  -s to specify the size of the square to compute (default 2, i.e. size 4 by 4)
```

Warning!

The `-o` option makes the program to write the image and histogram to disk always using with the *exact same* name. Therefore, rename them if you want to keep them and later compare them with your future parallel outputs. Both of these files are stored in binary format so that you will need to use `cmp` or `diff` commands to compare them.

1.3 Compilation

Look at the `Makefile` to understand how to compile each executable.

1.4 Execution

We have provided you with a set of scripts to analyse your parallel implementations.

1.5 Analysis

We suggest you to review the Lab 1 sections and annexes to remember the functionality of `Modelfactors` and how to use it to generate and analyse your OpenMP implementations. Remember also to look at your Lab 3 deliverable to remember everything you did.

Chapter 2

Iterative task decomposition

The iterative “tile” task decomposition pseudocode of the Mandelbrot set computation is shown in Code:1.

```
for (int y=0; y<numROW ; y+=TILE) {
    for (int x=0; x<numCOL; x+=TILE) {
        #pragma omp task
        {
            equal = ...           // Check if vertical TILE borders have the same colour
            equal = equal & ...    // Check if horizontal TILE borders have the same colour
            if (equal) {           // Fill the full TILE with the same colour
                fill_tile_code     // (if -d, display, if -h, update histogram)
            } else {               // Compute the full TILE in the Mandelbrot set
                compute_tile_code // (if -d, display, if -h, update histogram)
            }
        }
    }
}
```

Code 1: Iterative tile version of the parallel code computing the Mandelbrot set.

2.1 Implementation

We ask you to carry out the following actions:

Original tile iterative task decomposition. *Complete* the iterative parallel code that we have provided to you so that it exploits the parallelism and takes into account the dependencies/data sharings that you analysed in the Lab 3 in order to avoid concurrency problems. This program corresponds to the OpenMP version of the “original tile strategy” you analysed with Tareador in the previous laboratory.

Fine grain iterative task decomposition. *Implement the OpenMP version* of the “iterative fine grain parallel strategy” that you analysed with Tareador in Lab 3, which exploited the parallelism between the checks of the vertical and horizontal borders as well as between the full computation of the tile and the fill of the tile with the same colour. You should start with the completed version of the previous decomposition “original tile iterative task decomposition” as the baseline of this new version, including its task.

Important!

When starting with the fine grain version, do not remove the task from the previous version.

2.2 Execution

Run both the sequential and parallel codes in order to analyse them:

1. Run the sequential code with the following parameters to generate a histogram (`mandel_histogram.out`) and an image (`mandel_image.jpg`) as outputs:

```
sbatch submit-seq.sh mandel-iter -h -o -i 10000
```

2. Rename the outputs associated with the sequential code.
3. Run a parallel code with 20 threads to generate a histogram (`mandel_histogram.out`) and an image (`mandel_image.jpg`) as outputs:

```
sbatch submit-omp.sh mandel-iter 20
```

2.3 Correctness

Compare the two output files:

- the histogram `mandel_histogram.out`,
- the image `mandel_image.jpg`,

between the iterative sequential and the parallel versions. You can use `cmp` or `diff` Linux commands. Should these files differ, you will have to review your code and modify it accordingly.

2.4 Performance analysis

We ask you to run the Modelfactors analysis, analyse the Paraver trace with 16 threads, and perform a strong scaling analysis by performing the following steps:

1. Execute the Modelfactors analysis:

```
sbatch submit-strong-extrae.sh mandel-omp-iter
```

Look at the Lab document of the last session of Lab 1 (previous session to start Lab 4) to remember how to read and understand the Modelfactors tables.

2. Open the execution trace corresponding to 16 threads. Look again at the Lab 1 document to remember how to open various Paraver hints.
3. Perform a strong scalability test:

- (a) You may have to edit the `submit-strong-omp.sh` script and set the `SEQ` and `PROG` variables.
- (b) Run the following script:

```
sbatch submit-strong-omp.sh
```

We recommend you to take into account all the analysis done in Lab 3.

Chapter 3

Recursive task decomposition

The recursive “leaf” task decomposition pseudocode of the Mandelbrot set computation is shown in Figure 2.

```
recursive_call(matrix, tile) {
    equal = ...           // Check if vertical borders have the same colour
    equal = equal & ...   // Check if horizontal borders have the same colour
    if (equal) {
        #pragma omp task // Fill the full matrix tile with the same colour
        fill_tile_code   // (if -d, display, if -h, update histogram)
    } else {
        if (matrix_tile_size < TILE) {
            #pragma omp task // Computation of matrix tile (Mandelbrot set)
            compute_tile_code // (if -d, display, if -h, update histogram)
        } else {
            recursive_call(left-top sub-tile of matrix tile)
            recursive_call(left-bottom sub-tile of matrix tile)
            recursive_call(right-top sub-tile of matrix tile)
            recursive_call(right-bottom sub-tile of matrix tile)
        }
    }
}
```

Code 2: Leaf recursive version of the recursive parallel code computing the Mandelbrot set.

3.1 Implementation

We ask you to carry out the following actions:

Leaf recursive task decomposition. *Complete* the recursive leaf parallel code that we have provided to you so that it exploits some parallelism and takes into account the dependencies/data sharings that you analysed in the Lab 3 in order to avoid concurrency problems. This program corresponds to the OpenMP version of the “leaf strategy” you analysed with Tareador in the previous laboratory. Then, you should improve the completed version so that it reduces the task creation and synchronisation overheads and achieves speedups larger than 2 for more than 2 threads.

Tree recursive task decomposition. *Implement the OpenMP version* of the “recursive tree task decomposition parallel strategy” that you analysed with Tareador in Lab 3.

3.2 Execution

Run both the sequential and parallel code in order to analyse them:

1. Run the sequential code to only measure its execution time:

```
sbatch submit-seq.sh mandel-seq-rec -h -o -i 10000
```

2. Run the parallel code to generate a histogram (`mandel_histogram.out`) and an image (`mandel_image.jpg`) as outputs:

```
sbatch submit-omp.sh mandel-omp-rec 20
```

3.3 Correctness

Compare the two output files:

- the histogram `mandel_histogram.out`,
- the image `mandel_image.jpg`,

between the recursive sequential and the parallel versions. You can use `cmp` or `diff` Linux commands. Should these files differ, you will have to review your code and modify it accordingly.

3.4 Performance Analysis

We ask you to run the Modelfactors analysis, analyse the Paraver trace with 16 threads, and perform a strong scaling analysis by performing the same steps described in Section 2.4.

Appendix A

Laboratory deliverable information

A.1 Individual results for each parallel strategy

For each strategy that you analyse in this laboratory, Table A.1 explains what should be included in the report.

Table A.1: Analysis to be included in the PDF deliverable report for each parallel strategy.

Section	Description
Task decomposition	Name of the parallel strategy.
Code	Add the parallel source code to the compressed file. Indicate the name of the source code in the PDF. Please refrain from including the entire code in the deliverable document, except for fragments of code that you consider necessary in order to explain your work.
Modelfactor analysis	Include the Modelfactors tables to the report. In the case of “original tile iterative” and “leaf recursive” task decomposition implementations, you should identify the main issues that you observe based on the Modelfactors results. Then, in the case of “leaf recursive” you should also include the Modelfactors tables after the improvement we asked for. In the case of the “fine grain” iterative and “tree recursive” task decomposition implementations, you should compare them to the “original tile iterative” and “improved leaf recursive” task decomposition implementations, respectively, and write down all the improvements achieved. Remember that the Modelfactors analysis generates a directory with a set of execution traces to include in the corresponding table entry.
Paraver analysis	Include the Paraver views of the execution trace with 16 threads using the following hints: <code>Useful/Instantaneous parallelism</code> and <code>OpenMP tasking/explicit tasks function created and executed</code> , and comment them. In the case of the “leaf recursive”, you should include both non improved and improved versions. In the case of the “fine grain” iterative task decomposition, you should compare the execution trace with that of the “tile iterative” task decomposition. For the “tree recursive” task decomposition, you should compare it with the “improved leaf recursive” task decomposition.
Strong scalability	Include the scalability plot to the report as well as an explanation of the main aspects derived from your results. In the case of the “fine grain iterative” task decomposition, you should compare the execution trace with that of the “tile iterative”. For the “tree recursive” task decomposition, you should compare it with the “improved leaf recursive”.

A.2 Final comparison between all the parallel strategies

Table A.2 should be included after the description of all the parallel strategies. Note that you have to fill in this table using the output files generated after the execution of the `submit-strong-omp.sh`

script. Based on your performance results, explain which implementation you consider is the best one for the iterative and recursive versions.

Table A.2: Summary of the elapsed execution times for each of the versions, obtained from the output files after the execution of `submit-strong-omp.sh` script, that you need to include in your PDF deliverable report.

	Number of threads					
Version	1	4	8	12	16	32
Iterative: Tile (provided, but to be completed)						
Iterative: Fine grain						
Recursive: Leaf (provided, but to be completed)						
Recursive: Improved Leaf						
Recursive: Tree						
Best parallel implementation	Version	Reason why				

A.3 Evaluation of English as a third language

As you should know, this course contributes to the transversal competence “Tercera llengua”. Please deliver your material in English if you want this competence to be evaluated and also refer to the “Rubrics for the third language competence evaluation” document to know which rubrics will be utilised.

A.4 Submission of the deliverable and source codes to Atenea

An entry for the submission will be created in Atenea and the proper submission deadlines will be set. Only a PDF format for this document will be accepted for submission. Additionally, you are required to submit the OpenMP source codes that you have completed and developed. Please refrain from including the entire source code in the deliverable document, except for fragments of codes that you consider necessary to explain your work.

Important!

You must upload two separate files to Atenea:

1. The deliverable document in PDF format.
2. A compressed file (zip or tar.gz) containing all the requested source codes.